



*Codice sotto la lente:
la mia esperienza con Codex di OpenAI*

Sommario

Codice sotto la lente: la mia esperienza con Codex di OpenAI.....	3
1. Cos'è Codex e perché è diverso da “una semplice AI che scrive codice”	3
2. Due modi di usare Codex: dentro ChatGPT o direttamente nell'IDE	3
3. Prerequisiti per utilizzare Codex.....	4
4. Modalità di accesso e di utilizzo.....	4
5. Il caso d'uso concreto: un notebook Jupyter per la previsione dei consumi elettrici	6
6. La modalità ASK: analisi, review e report	8
7. Dall'analisi alla modifica: la modalità EDIT	9
8. La modalità Agent: autonomia totale (con cautela).....	11
9. I modelli GPT supportati in VS Code.....	11
10. Diverse modalità di risposta fornite dai vari modelli	11
11. Considerazioni finali	12

Codice sotto la lente: la mia esperienza con Codex di OpenAI

Negli ultimi due anni ho avuto modo di sperimentare diversi modelli di intelligenza artificiale applicati allo sviluppo software e alla data science. Alcuni promettevano molto, altri offrivano risposte interessanti ma poco affidabili quando il codice diventava reale, complesso e strutturato. Per questo, quando ho iniziato a utilizzare **Codex**, la nuova funzionalità di **OpenAI** dedicata al codice, mi sono posto una domanda semplice ma cruciale: posso fidarmi di questo strumento come mi sono fidato, finora, di ChatGPT? Dopo un utilizzo concreto, basato su casi reali e non dimostrativi, la risposta è stata positiva, e proprio questa esperienza voglio condividere.

In questo documento mi concentrerò esclusivamente sull'analisi delle modalità operative e d'uso di Codex, mostrando come interagire con i notebook, con l'IDE e con le varie opzioni di revisione e modifica del codice. Nei prossimi giorni, invece, dedicherò un articolo più approfondito, che pubblicherò nella mia newsletter "*Codice sostenibile*", in cui entrerò nel dettaglio delle review operate da Codex, illustrando le specifiche modifiche apportate e come queste possano influire su affidabilità, performance e manutenibilità del codice.

1. Cos'è Codex e perché è diverso da "una semplice AI che scrive codice"

Codex non è un nuovo modello separato da GPT, né un prodotto a sé stante. È meglio pensarlo come **una modalità specializzata di utilizzo dei modelli GPT di OpenAI**, progettata per lavorare sul codice in modo strutturato, consapevole e contestuale.

Se ChatGPT, fin dall'inizio, si è dimostrato molto efficace nel generare e spiegare codice, Codex fa un passo in più: non si limita a "scrivere", ma **legge, analizza, valuta e propone modifiche** a programmi reali, tenendo conto dell'intero file o addirittura di un notebook completo. La differenza chiave sta proprio qui: Codex non ragiona su frammenti isolati, ma sul **contesto complessivo del codice**, ed è questo che lo rende particolarmente adatto a scenari professionali come la data science, il machine learning o lo sviluppo software strutturato.

2. Due modi di usare Codex: dentro ChatGPT o direttamente nell'IDE

Codex può essere utilizzato in due contesti distinti, che rispondono a esigenze diverse. Il primo è **all'interno di ChatGPT**, direttamente dal browser. In questo caso Codex viene utilizzato come revisore o consulente: si incolla il codice, si chiede un'analisi, una spiegazione o un miglioramento, e si ottiene una risposta dettagliata. È un approccio utile per ragionare su singoli script o funzioni, ma mostra alcuni limiti quando il codice cresce in dimensione o complessità. Il secondo, ed è quello che ho trovato davvero efficace, è **l'integrazione diretta nell'IDE**, nel mio caso Visual Studio Code (VSCode). Qui Codex entra nel flusso di lavoro quotidiano dello sviluppatore e smette di essere

un “assistente esterno” per diventare una vera e propria **supervisione intelligente del codice**.

3. Prerequisiti per utilizzare Codex

Uno degli aspetti più interessanti è che Codex non richiede infrastrutture particolari. I prerequisiti sono relativamente semplici.

Serve innanzitutto un **account OpenAI attivo**, lo stesso che permette di utilizzare ChatGPT. Non esiste un account Codex separato. Se si ha accesso ai modelli GPT più avanzati, Codex è già disponibile.

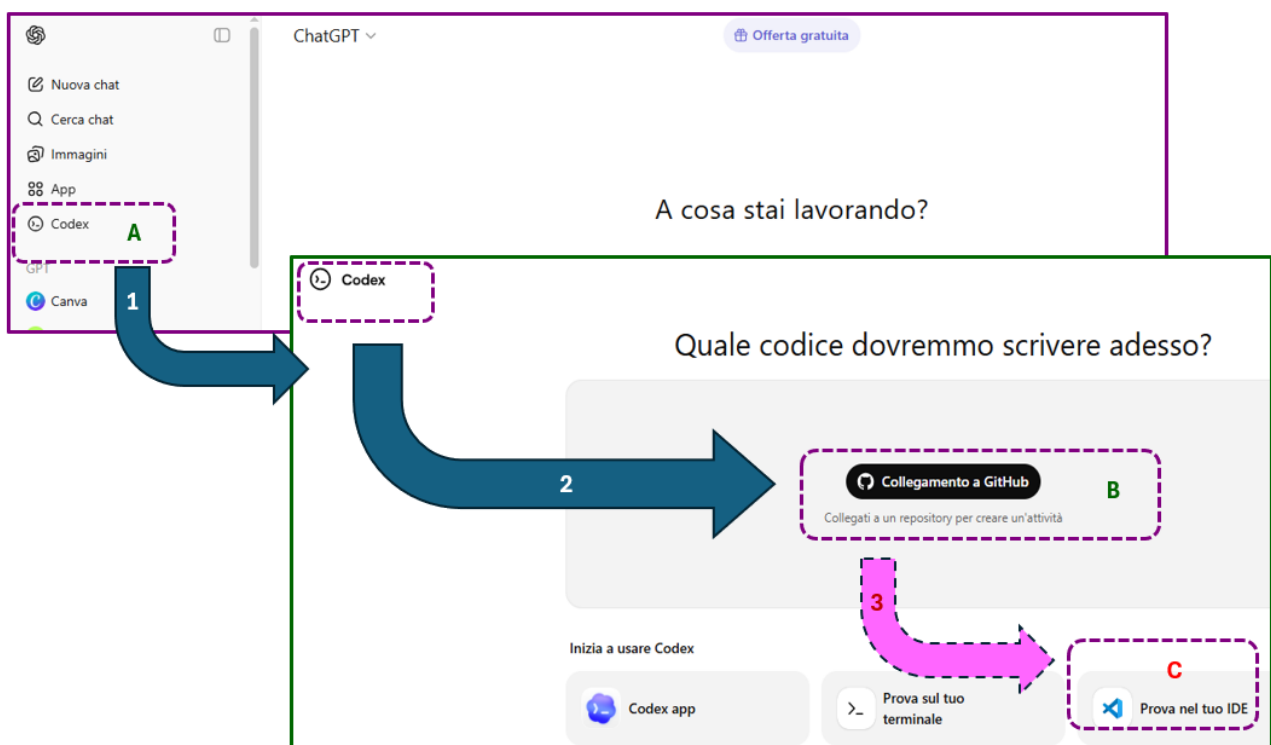
Per l'utilizzo nell'IDE sono necessari alcuni elementi aggiuntivi:

- Visual Studio Code aggiornato,
- un account GitHub
- e le estensioni GitHub Copilot con Copilot Chat.

GitHub, in questo contesto, non serve per pubblicare codice o lavorare su repository pubblici, ma funge semplicemente da **meccanismo di autenticazione**. Tutto il codice resta locale e sotto il pieno controllo dell'utente.

4. Modalità di accesso e di utilizzo

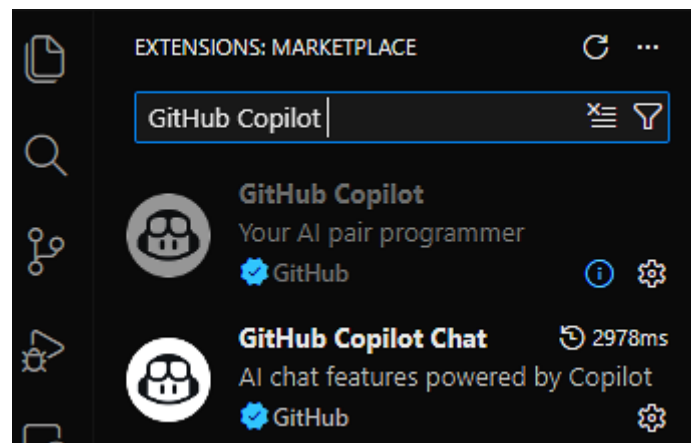
Nella figura di sotto evidenzio i tre passi principali per poter accedere a Codex direttamente da VsCode.



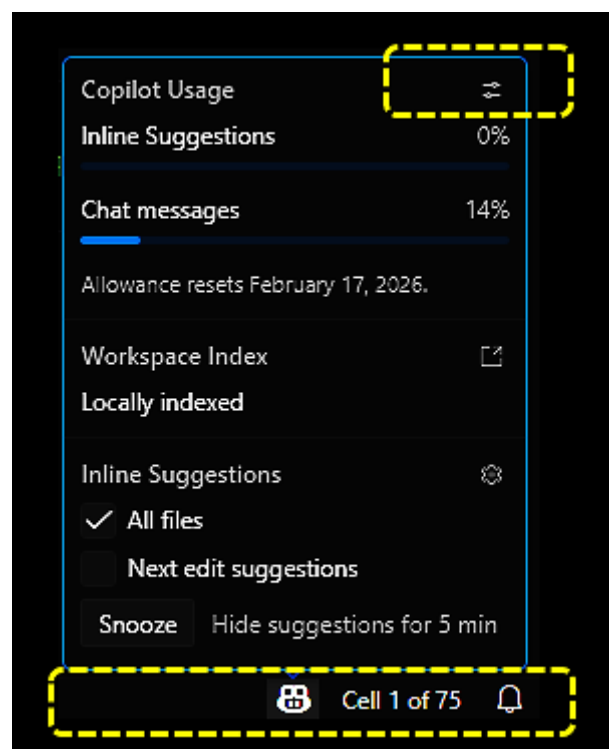
Innanzitutto da ChatGpt richiamo la modalità Codex (in alto a destra **step A**). Successivamente dopo aver avuto accesso a Codex, effettuo il login al mio account GitHub (step B) e dopo l'autenticazione richiamo VsCode dal mio PC.

NB: Per il momento non ho richiesto l'utilizzo la modalità di "Agente automatico" richiamabile attraverso la funzione "Prova il tuo IDE" (Step C) che analizzerò in dettaglio in seguito.

All'interno del mio ambiente VsCode devono essere attivi le due estensioni GitHub Copilot e GitHub Copilot Chat...



... in tal caso in basso a destra cliccando sull'icona di GitHub è possibile controllare il collegamento a questo attraverso le due frecce come indicato in figura...



L'ambiente è configurato correttamente e si può passare allo step successivo.

5. Il caso d'uso concreto: un notebook Jupyter per la previsione dei consumi elettrici

Il mio primo utilizzo reale di Codex in VS Code ha riguardato un notebook Jupyter scritto in Python per la previsione dei consumi elettrici (*Previsione Fabbisogno.ipynb*). Non un esempio didattico, ma un notebook completo, con fasi di preprocessing, feature engineering, addestramento del modello e valutazione dei risultati.

Il notebook era già caricato in VS Code. A questo punto, tramite **Copilot Chat**, ho selezionato l'intero file e ho scelto la modalità **ASK**, ovvero la modalità di analisi.

Questa scelta è fondamentale: ASK significa “analizza e suggerisci”, non “modifica”. In altre parole, Codex assume il ruolo di un revisore esperto che legge tutto il codice e fornisce un parere strutturato, senza toccare una singola riga del programma originale.

Per fare ciò ho eseguito i seguenti passi :

1. Per attivare *Copilot Chat* : **CTRL+Shift+P** (si apre il pop up) e digitare “*Copilot Chat: Open Chat*”, poi premere invio.
2. Una volta aperta la finestra di dialogo per la chat come indicato nella figura di sotto...



3. Dovrebbe comparire in automatico il file/programma su cui si sta lavorando. In caso contrario caricarlo (step 1);
4. Inserire il prompt (che vedremo in dettaglio dopo) verso il copilota (step 2);
5. Selezionare la modalità di interazione che nel nostro caso è "ASK" (step 3);
6. Selezionare il modello con cui operare; nel nostro caso è GPT-4o (step 4)

In particolare, il prompt inserito (step 2) è stato il seguente :

[MODALITÀ 2 – REVIEW MODE / READ-ONLY – CODE-LEVEL ANALYSIS]

Analizza integralmente il notebook Jupyter selezionato come se dovessi effettuare una code review tecnica forense.

 **IMPORTANTE:**

NON limitarti a descrivere rischi teorici o best practice. OGNI problema segnalato deve essere dimostrato direttamente sul codice esistente.

Per ogni criticità individuata (bug, errore metodologico, inefficienza):

1. Identifica ESATTAMENTE:

- la cella coinvolta
- le variabili specifiche
- il flusso dei dati (da dove arrivano e dove finiscono)

2. Riporta SEMPRE:

- il codice originale rilevante (copiato dal notebook)
- una spiegazione concreta del perché quel codice produce il problema (non "potrebbe", ma "produce")

3. Se il problema è reale:

- fornisci una versione corretta e funzionante del codice
- scrivi il codice COMPLETO della sezione modificata
- etichettalo chiaramente come:

"CODICE PROPOSTO – NON APPLICATO"

4. Se NON trovi evidenze concrete nel codice:

- dichiara esplicitamente:

"Nessuna evidenza concreta di problema nel codice analizzato"

Output:

- genera un REPORT DI REVIEW in formato Markdown
- il report deve contenere sezioni separate:
- Analisi

- *Evidenze nel codice*
- *Impatto sul modello*
- *Codice originale*
- *Codice corretto (non applicato)*

Vincoli assoluti:

- *NON modificare il notebook*
- *NON riscrivere celle*
- *NON applicare patch*
- *NON eseguire codice*
- *NON usare formulazioni ipotetiche in assenza di evidenze.*

A questo punto ho inviato la mia richiesta ed il copilota ha effettuato la sua analisi.

6. La modalità ASK: analisi, review e report

In modalità ASK ho chiesto a Codex di effettuare una review completa del notebook e di generare un report strutturato. Il risultato è stato sorprendentemente solido.

Il report conteneva, per ogni sezione rilevante:

- il codice originale analizzato
- l'individuazione di potenziali errori, warning o miglioramenti possibili
- la spiegazione del perché quella parte poteva essere migliorata
- una versione di codice corretta o ottimizzata

Il punto fondamentale è che **nulla è stato applicato automaticamente**. Tutto è rimasto nel report, pronto per essere valutato, discusso o ignorato. In questo senso Codex si comporta come un revisore senior: propone, motiva, ma non impone.

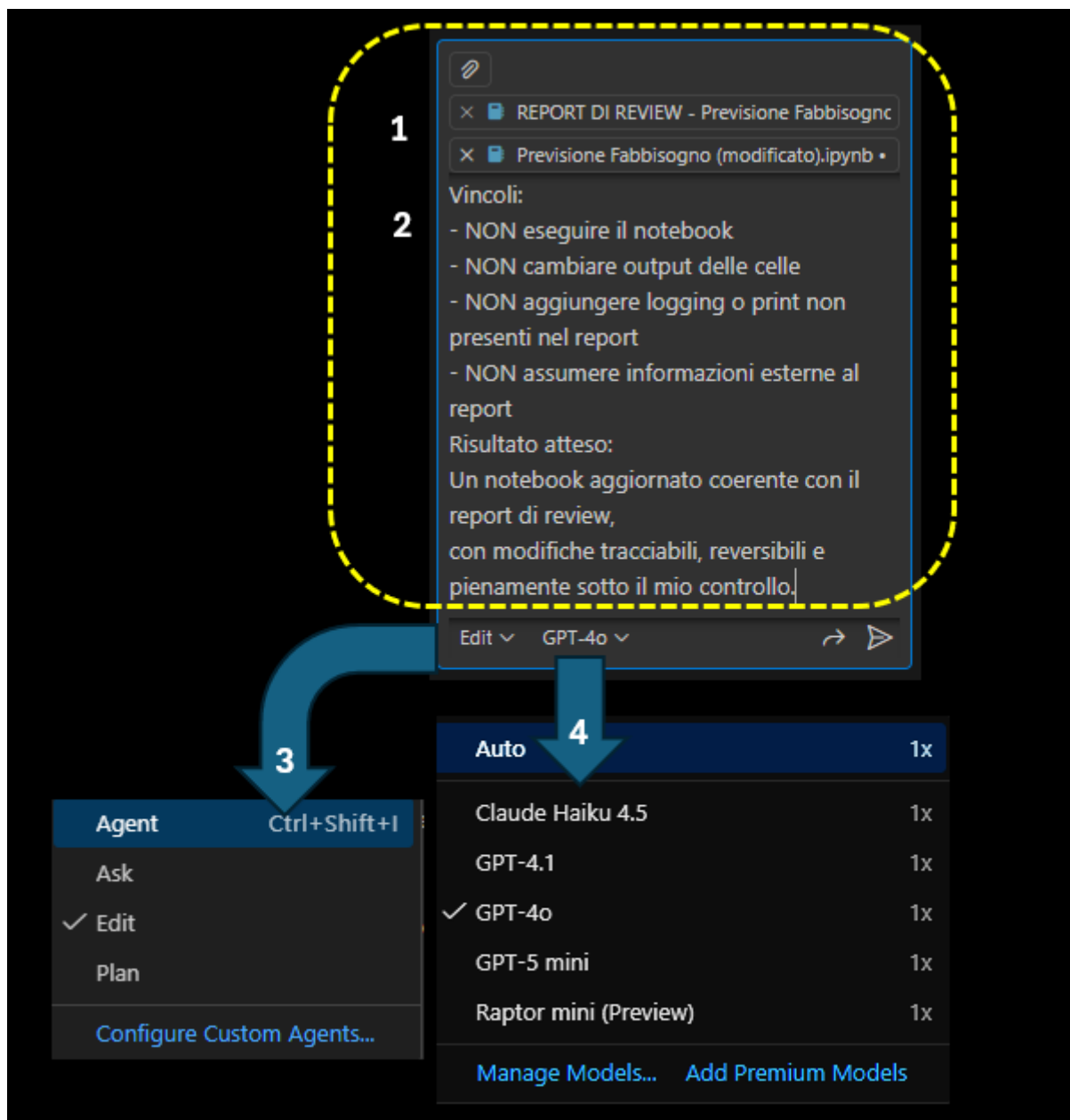
In allegato il report prodotto in formato markdown.



REPORT DI REVIEW
- Previsione Fabbiso

7. Dall'analisi alla modifica: la modalità EDIT

Una volta analizzato il report, ho deciso di applicare le modifiche suggerite. Qui entra in gioco la modalità **EDIT**. La figura di sotto sintetizza il flusso previsto.



Passare da ASK a EDIT significa autorizzare Codex a intervenire direttamente sul codice, ma sempre in modo controllato. Codex applica le modifiche una per una e VS Code mostra chiaramente ogni cambiamento, evidenziandolo nel notebook. Per ciascuna modifica è possibile scegliere se mantenerla (**Keep**) o annullarla (**Undo**).

Nel mio caso le modifiche erano numerose, ma coerenti. Ho potuto accettarle tutte, salvare il notebook e ritrovarmi con una versione migliorata del codice, senza aver perso il controllo in nessun momento. Questo passaggio è cruciale: Codex non lavora "in background", ma **rende ogni intervento trasparente e reversibile**.

Il prompt che ho utilizzato è stato :

[MODALITÀ EDIT – APPLY FIXES FROM REVIEW]

Contesto:

È già stato generato un REPORT DI REVIEW dettagliato sul notebook Jupyter attualmente aperto.

Il report contiene:

- analisi forense del codice*
- identificazione puntuale dei problemi*
- sezioni di "CODICE PROPOSTO – NON APPLICATO"*

Obiettivo:

Applicare al notebook ESCLUSIVAMENTE le modifiche presenti nel report precedente, utilizzando solo il codice indicato come corretto.

Istruzioni operative:

- applica solo le modifiche esplicitamente proposte nel report*
- NON introdurre nuove ottimizzazioni o refactoring non presenti nel report*
- NON modificare parti di codice non coinvolte nei problemi analizzati*
- mantieni invariata la struttura generale del notebook (celle, ordine, commenti esistenti)*
- se una modifica richiede la sostituzione di codice:*
 - sostituisci integralmente la sezione originale con il codice corretto*
- se una modifica è ambigua o non applicabile direttamente:*
 - NON applicarla*
 - segnala il motivo nel commento della modifica*

Modalità di applicazione:

- applica le modifiche in modo incrementale*
- evidenzia ogni modifica come singola proposta (diff)*
- richiedi conferma esplicita tramite "Keep / Undo" per ciascuna modifica*

Vincoli:

- NON eseguire il notebook*
- NON cambiare output delle celle*
- NON aggiungere logging o print non presenti nel report*
- NON assumere informazioni esterne al report*

Risultato atteso:

*Un notebook aggiornato coerente con il report di review,
con modifiche tracciabili, reversibili e pienamente sotto il mio controllo.*

Il copilot GitHub mi ha rilasciato il codice aggiornato che allego:



Previsione
Fabbisogno (modifi

8. La modalità Agent: autonomia totale (con cautela)

Esiste infine la modalità **Agent**, che rappresenta il livello massimo di autonomia. In questa modalità Codex non si limita ad analizzare o modificare singole parti, ma può pianificare ed eseguire una sequenza di azioni: leggere il codice, decidere cosa migliorare, applicare le modifiche e iterare.

È una modalità potente, pensata per attività ripetitive o per refactor ben definiti, ma va utilizzata con consapevolezza. Personalmente non l'ho ancora usata, ma la considero adatta a contesti in cui il problema è ben delimitato e l'obiettivo chiaro.

9. I modelli GPT supportati in VS Code

All'interno di VS Code è possibile scegliere il modello da utilizzare. Le opzioni includono la selezione automatica oppure modelli specifici come GPT-4.0, GPT-4.1 e GPT 5 mini di OpenAI o Claude Haiku 4.5. Per i primi 3 si tratta di **modelli OpenAI**, ottimizzati per il codice e per il reasoning tecnico. Questo aspetto è stato determinante per me: sapere che la qualità di analisi resta allineata a quella di ChatGPT, ma con un contesto di lavoro molto più ricco.

10. Diverse modalità di risposta fornite dai vari modelli

La prima volta che ho utilizzato **GPT-4o** l'ho fatto in modalità **Auto**, senza alcun prompt specifico che indirizzasse il comportamento del modello. In questa modalità, a parte qualche rara eccezione in cui mi ha indicato il codice da sostituire o migliorare, per lo più il modello si è limitato a suggerire strategie, ossia a spiegarmi *cosa fare* senza fornire il codice concreto. Al contrario, **Claude** è stato immediatamente più operativo, fornendomi il codice da sostituire in modo chiaro e dettagliato.

Questo comportamento non deve sorprendere: dipende dal modo in cui i modelli sono addestrati e dal ruolo che assumono implicitamente quando analizzano il codice. Non si tratta di differenze di qualità, ma di approccio. **Claude tende a comportarsi come un pair programmer molto operativo**: quando individua un problema prende subito una

decisione su come risolverlo, scrive il codice sostitutivo, e lo fa senza timore di entrare nei dettagli. È meno cauto nel vincolarsi al contesto progettuale, privilegiando la concretezza e la completezza rispetto alla discussione delle alternative. Il risultato è immediato: davanti a un errore o a una criticità, Claude non si limita a indicare il problema, ma propone direttamente la soluzione pronta all'uso.

GPT-4o, invece, si comporta spesso come un reviewer architetturale. Analizza attentamente il codice, evidenzia il motivo per cui una sezione è problematica, distingue tra errore logico, limite strutturale e debito tecnico, e suggerisce strategie di correzione piuttosto che imporre un singolo codice corretto. È un approccio più prudente e analitico: il modello ti dice cosa fare, ti guida nella decisione, ma lascia a te la scelta finale di implementazione.

Quando si utilizza la modalità Auto, questa differenza si amplifica. Claude interpreta la modalità Auto come un invito ad agire e risolvere subito i problemi, mentre GPT-4o interpreta Auto come una modalità di analisi, consigliando senza modificare nulla. Tuttavia, questa differenza può essere superata: con un [prompt specifico](#) che richieda a GPT-4o di generare direttamente il codice corretto completo, il modello produce risultati eccellenti, molto simili a quelli di Claude. In altre parole, la differenza osservata inizialmente non è dovuta a una limitazione intrinseca, ma al modo in cui il modello viene guidato tramite il prompt.

Quindi, quale modello conviene usare? La risposta dipende dalla fase in cui ci si trova nel workflow. Claude è particolarmente utile quando si desidera un output immediato, quando si vuole codice pronto da applicare, o quando si stanno facendo *refactoring locali* e si ha già chiara la direzione da seguire. In questo senso, Claude funziona come un “sviluppatore esecutivo”: rapido, operativo e concreto. GPT-4o, invece, è più adatto quando serve una review seria, quando si vuole capire se una soluzione è realmente corretta, o quando si lavora su codice complesso e concettualmente delicato. Funziona come un “senior reviewer”, più analitico e prudente, ideale per evitare correzioni che potrebbero funzionare a breve ma non essere sostenibili o scalabili nel tempo.

La cosa interessante è che, **con il giusto prompt, GPT-4o può fornire risultati altrettanto concreti e dettagliati come Claude**, rendendolo molto versatile nel workflow di revisione e correzione del codice. La differenza iniziale tra i due modelli non era quindi una questione di capacità, ma di impostazione e di guida fornita: una scelta consapevole del prompt può trasformare l'approccio prudente di GPT-4o in un'analisi tanto operativa quanto precisa.

11. Considerazioni finali

Dopo aver utilizzato Codex in un contesto reale, posso dire che non si tratta di “un'altra AI che scrive codice”, ma di uno **strumento di supervisione intelligente**, capace di affiancare lo sviluppatore senza sostituirlo.

Il valore di Codex non sta nell'automazione fine a sé stessa, ma nella capacità di **migliorare il codice mantenendo l'umano al centro delle decisioni**. È questo equilibrio, raro, che lo rende uno strumento affidabile per chi lavora quotidianamente con codice reale e complesso.

Per chi, come me, negli ultimi anni ha cercato un supporto AI davvero utile e coerente, Codex rappresenta un passo avanti concreto. Non perfetto, ma maturo. E soprattutto, utilizzabile oggi.